

Strapi as a content CMS for Dearly's

Feasibility, cost, and implementation plan — content only, commerce untouched

Prepared 19 September 2026 · Scope: Strapi owns marketing content. The Express + MongoDB backend keeps all commerce data and is not modified.

Verdict: feasible, cheap and low risk. Your MongoDB is not affected.

Strapi keeps its own small database for its own content and never connects to MongoDB. The two systems share no data, so nothing about your Mongoose models, orders or payments constrains this. Community Edition is free. Estimated effort is 4–6 days, and nothing existing is put at risk.

Why MongoDB does not matter here

Strapi always manages its own database — SQLite by default, in a single file, with Postgres as the production option. The documentation is explicit that a Strapi application is not connected to a pre-existing database. That is usually stated as a limitation, but under this scope it is the reason the question disappears: Strapi never reaches into your data at all.

System	Owns	Database	Changes
Express API existing	Products, categories, cart, orders, payments, coupons, reviews, users, addresses	MongoDB Atlas	None
Strapi new	Announcements, hero slides, promo bands, testimonials, journal posts, USP strip, makers, footer columns	Its own SQLite or Postgres	New install
Next.js	Renders both	—	One new API client

The only coupling is in the browser: Next.js fetches commerce from Express and content from Strapi. Neither server talks to the other. There is no shared schema, no join, no migration, and no cross-database reference, because no record in one system points at a record in the other.

Correction to the previous version of this report.

The earlier assessment treated MongoDB as a blocker. That applied only to a scope in which Strapi took over the product catalog, which was an assumption made in that report rather than a requirement. For content-only adoption it does not apply, and the recommendation reverses from "do not adopt" to "reasonable to adopt".

What Strapi would actually own

The case for this is already visible in the repository. `src/data/site.ts` is 296 lines of hardcoded marketing content, every item of it rendered on the live site, and every change to it currently needs a developer, a commit and a deploy.

Export	Renders in	Strapi content type
announcements	AnnouncementBar.tsx	Collection — rotating banner messages
heroSlides	Hero.tsx	Collection — homepage carousel
promoBand	Sections.tsx	Single type — promotional strip

Export	Renders in	Strapi content type
testimonials	Editorial.tsx	Collection — customer quotes
journalPosts	Editorial.tsx	Collection — editorial / blog
uspStrip	Editorial.tsx	Collection — value propositions
makers	Sections.tsx, Footer.tsx	Collection — maker profiles
recipients	Sections.tsx	Collection — gift-recipient tiles
footerColumns	Footer.tsx	Single type — footer navigation

Deliberately left in code

- brand and navigation — structural, referenced across 10+ components including the admin shell and checkout. Editing these through a CMS invites someone breaking site navigation with no review.
- shippingMethods, taxRate, freeShippingThreshold, coupons — these are pricing rules, not content. They belong with commerce logic, and real coupons already live in MongoDB.
- products.ts and taxonomy.ts — demo catalog fallbacks. The live catalog is already served from MongoDB. Moving these into Strapi is the expensive scope this report advises against.

Cost

Item	Cost	Note
Strapi Community Edition	Free, MIT licence	REST and GraphQL, role-based access control, unlimited entries and API calls, media library
Database	\$0 – \$15/mo	SQLite is free and file-based. A small managed Postgres is advisable once editors rely on it
Hosting	\$5 – \$20/mo	One small Node container; Strapi Cloud is optional, not required
Paid Strapi tiers	Not needed	Growth at \$45/mo adds Live Preview, Releases and Content History. None are required to launch
Ongoing total	roughly \$5–35/mo	Plus 4–6 days of engineering time once

Implementation plan

Phase 1 — Install and model, 1.5 days

- `npx create-strapi-app@latest cms --quickstart` — SQLite, no database setup.
- Create the nine content types above, matching the field names already used in `site.ts` so the frontend mapping stays trivial.
- Upload existing imagery into the media library, or point the upload provider at your Cloudinary account so URLs stay consistent with the product images.
- Set the public role to read-only and generate an API token for server-side fetches.

Phase 2 — Seed the content, 1 day

- Write a one-off script that reads `site.ts` and POSTs each entry into Strapi, so the site launches with exactly the content it has today and nothing needs retyping.

- Verify each collection renders identically to the current hardcoded version.

Phase 3 — Wire the frontend, 1.5 days

- Add `src/lib/cms-config.ts` and `src/lib/cms.ts`, mirroring the existing `api-config.ts` and `api.ts` split, pointing at `NEXT_PUBLIC_CMS_URL`.
- Fetch content in Server Components with ISR, the same pattern `catalogServer` already uses. Content changes then appear without a deploy, and the browser makes no extra request.
- Swap each `site.ts` import for its CMS call, one component at a time. Keep the hardcoded arrays as the fallback when a fetch fails, so a CMS outage degrades to today's site rather than an empty page.
- Leave every commerce client — `cartApi`, `orderApi`, `paymentApi`, `catalogApi` — completely untouched.

Phase 4 — Deploy and hand over, 1 day

- Deploy Strapi beside the API; add `NEXT_PUBLIC_CMS_URL` to the frontend environment.
- Create editor accounts and walk marketing through publishing one banner end to end.
- Confirm the ISR revalidation window matches how quickly they expect edits to appear.

Phase	Days	Risk if abandoned midway
1 — Install and model	1.5	None; nothing in the app has changed yet
2 — Seed content	1.0	None; Strapi is still unused by the site
3 — Frontend wiring	1.5	Low; fallbacks keep the current content rendering
4 — Deploy and hand over	1.0	Low; revert one environment variable
Total	5 days	<i>Reversible at every phase</i>

Risks

Risk	Severity	Mitigation
A second service to deploy, back up and patch	Medium	The standing cost of a CMS; unavoidable, and small at this scale
CMS downtime blanks homepage sections	Low	Keep the <code>site.ts</code> arrays as fallbacks; ISR also serves the last good render
Editors change copy that breaks a layout	Low	Set max-length on text fields; the components already truncate
SQLite outgrown once several editors work concurrently	Low	Move to managed Postgres; a supported, documented migration
Strapi major-version upgrades	Low	Pin the version. Content-only installs have a small upgrade surface

Recommendation

Go ahead, scoped to content.

This is the version of Strapi adoption that pays for itself: 296 lines of marketing copy stop being a developer task, commerce stays exactly where it is, and MongoDB is untouched and irrelevant. Hold the line firmly at products — the moment Strapi is asked to own the catalog, the database migration, the cross-store seam and the duplicated admin UI all return, and the case stops being worth it.

Sources

Database configuration and the pre-existing-database limitation — docs.strapi.io/cms/configurations/database

Supported databases — docs.strapi.io/snippets/supported-databases

Self-hosted pricing — strapi.io/pricing-self-hosted

Licensing — github.com/strapi/strapi/blob/main/LICENSE

Line counts and component usage measured directly from the `dearly-s` repository on 19 September 2026.